

Debug Progress Report

AI_judge

Debug Progress Report

Riku / Codex draft

2026-07-14

Document Control

Version	0.1 draft
Prepared By	Riku / Codex
Reviewers	TBD
Date Prepared	2026-07-14
Reporting Window	2026-07-14
Status	Draft / pre-detail

Purpose

This document is a first-pass debug progress report for AI_judge. It summarizes what appears to be complete, what is unstable, and which fixes should be considered first.

Detailed logs and screenshots can be added later. This draft uses only the README, visible source files, training log, and current working-tree state.

Business Context

AI_judge is an image-classification project that judges whether an image is real or AI-generated. The README indicates that preprocessing, ResNet18 transfer learning, metric logging, best-model saving, and a Streamlit app are already underway.

The current debug focus should be reproducibility: align training code, data paths, model artifact locations, and the app's model-loading path before deeper accuracy work.

Debug Summary

Field	Details
Project Name	AI_judge
Debug Key	Path / import / model artifact consistency
Owner Team	Personal project

Business Owner	Riku
Product Surface	Python training pipeline + Streamlit app
Primary Objective	Make the next repair order clear
Current Implementation	README, logs, Streamlit app, and ResNet code exist
Target State	Rerunnable with the same data, save path, and model definition
Allocation	
Unit of Randomization	N/A
Audience	Developer / later reviewer
Exclusions	Detailed logs, external eval data, and unrun tests
Start Date	TBD
End Date	Open
Planned Runtime	smoke check -> 1 epoch -> normal training
Actual Runtime	TBD

Design Notes

This review treats the current state as a debug inventory rather than a full code rewrite. The five main surfaces are data input, training execution, evaluation logs, model saving, and web inference.

Open items are kept as follow-up checks, not final conclusions. The real archive layout and the exact training path that produced the current model artifact still need confirmation.

Hypothesis

If path, import, and artifact-location inconsistencies are resolved first, training results and web inference can be compared under the same assumptions. That should make later accuracy and UI decisions easier.

Success Criteria

Recommended priority order: 1. blocking import/path issues, 2. model mismatch between training and inference, 3. reproducible evaluation logs, 4. accuracy improvements, 5. UI improvements.

Metric	Target/Rule
Primary Metric	`run.py`, dataloaders, and app run under one consistent setup
Guardrail 1	Do not revert current `model/My_CNN/dataset.py` work
Guardrail 2	Do not overwrite existing model and log artifacts
Guardrail 3	Record model, split, and preprocessing for comparisons
Decision Rule	Fix blockers first; improve accuracy afterward

Pre-Fix Validation

- Confirm that README progress matches the current files.
- Confirm which training code `run.py` calls and which dataset module actually exists.
- Confirm whether `loss.csv` and the saved model file belong to the same experiment.
- Confirm the model filename, class order, and checkpoint format expected by the Streamlit app.
- After fixes, validate in this order: dataloader only, one-epoch training, then app startup.

Current Progress and Evidence

The training log shows the best validation accuracy at epoch 18 (79.53%); epoch 20 ends at 79.10%. Together with the README, this suggests the baseline model and app shell have reached a working draft.

Metric	Current	Target / Next Check
Training log	best valid acc 79.53% at epoch 18	
Final epoch	train acc 79.96% / valid acc 79.10%	

Model artifact	`resnet18_real_fake.pth` in root	Align save path and app load path
Data path	`archive` exists	Check `train/valid` before execution
Smoke check	Not run	Check one epoch and Streamlit startup

Primary Outcome

Measure	Current	Target	Gap	Priority
Primary outcome	Inventory complete	Rerunnable execution path	import/path mismatch	High
95% Confidence Interval	N/A	N/A	N/A	N/A
Two-Sided p-value	N/A	N/A	N/A	N/A
Outcome Narrative	Training and app pieces exist	Read the same artifact	Save path/name	High

Primary progress: ResNet-family training logs exist, best-model saving is implemented, and a Streamlit inference screen is present. Primary risk: file names, save locations, and import names are not yet fully aligned.

Guardrail Checks

Guardrail Metric	Current	Target	Delta	Status
User changes	`dataset.py` modified	Preserve current work	Care needed	Watch
Model overwrite risk	Existing pth file	Save explicitly or under new name	Policy needed	Watch
Data availability	`archive` exists	Structure checked	Unverified	Open
App consistency	Loads ResNet18 pth	Matches training code	Needs check	Open

Guardrails for the next fix pass: do not overwrite existing training outputs, do not revert the user's current `model/My\_CNN/dataset.py` changes, and do not mix model-comparison conditions.

Debug Area Results

The review separates dataset handling, baseline training, My_CNN training, evaluation graphing, and the Streamlit app. Keeping these areas separate makes the next repair order clearer.

Area	Current Value	Target Value	Gap	Interpretation
README	Progress list exists	Synced with implementation	May be partly stale	Update candidate
baseline	ResNet50 code exists	Valid dataset import	`tensor` name needs check	Fix early
My_CNN	Custom CNN code exists	Valid dataset import	`tensor` name needs check	Fix early
graph.py	Reads loss.csv	Stable root-based path	BASE_DIR may be too deep	Fix candidate
app.py	Streamlit screen exists	Model/class order match training	Artifact origin check	Check candidate

The highest-value first fixes are import and path issues because they can block execution immediately. Accuracy and UI work are easier once the same artifacts can be read and written consistently.

Data Quality and Limitations

- `archive` exists, but this draft does not yet verify the full dataset contents.
- `loss.csv` is present, but its exact model definition and data split provenance still need confirmation.
- `model/baseline/ResNet50.py` imports `model.baseline.tensor`, while the visible source file is `dataset.py`.
- `model/My\_CNN/NN\_part1.py` also imports `model.My\_CNN.tensor`; that source-name mapping needs confirmation.
- The Streamlit app reads `resnet18\_real\_fake.pth` from the repository root, which differs from the ResNet50 output path.

Therefore, this is a configuration-based debug draft, not a final diagnosis with fresh execution logs.

Interpretation

The project does not look blocked so much as split across several experiment paths. Aligning references and artifacts should make the next improvement pass much easier.

Recommended sequence: fix import/path issues, align trained-model artifacts with the app loader, then proceed to ResNet50/EfficientNet comparisons and data augmentation.

Decision Log

Item	Decision
Final Decision	Prioritize execution-path stability before accuracy work
Decision Date	2026-07-14
Approvers	Riku
Rollout Type	Small fixes; verify dataloader -> training -> app
Rollback Trigger	Stop if a change could overwrite existing models or logs
Follow Up	Add detailed logs, then split into concrete fixes

Next Debug Actions

1. Choose whether dataset imports should use ``tensor`` or ``dataset``, then make the code consistent.
2. Add a simple existence check for ``archive/rvf10k/train`` and ``archive/rvf10k/valid``.
3. Choose one model artifact location: repository root or ``output/``.
4. Fix ``graph.py`` so ``loss.csv`` is resolved independently of the current working directory.
5. Use one-epoch training and Streamlit startup as the post-fix acceptance checks.

Appendix A: Debug Metric Definitions

- **Executability:** imports, data paths, and model-save locations are aligned and can be rerun with the same command.

- **Reproducibility:** `loss.csv`, model files, and `class\_to\_idx` are saved as one experiment unit.
- **Inference consistency:** the app uses the same class order, preprocessing, and model structure as training.
- **Accuracy-improvement readiness:** augmentation, fine-tuning, and model comparisons can be tested under the same evaluation setup.
- **UI readiness:** missing-model guidance, upload flow, confidence display, and caveats are clear to users.

Appendix B: Analyst Notes

- Reference `template` SHA-256:
d823cd0115186b34c01c6e4b4da3be28b64ee73cac849dbd62d6f4bb6385b0fb
- This version is a debug-structure sample before detailed logs are added.
- A later version can add commands, full error text, patches, and confirmation results.
- Current user changes should be preserved; fixes should be scoped and separated.